Hybrid Algorithm-Based 3D Container Loading Optimisation

Technical report / 9 November 2026

Objective

Maximize the sum of loaded package volumes in one rectangular container. A package may be omitted or placed once at an axis-aligned position in an allowed orientation. Every placement must fit within the internal bounds, avoid positive-volume overlap and keep the total loaded weight within the payload limit.

Representation and scoring

Geometry uses integer millimetres and mass uses integer grams. Each placement records the package ID, lower corner and oriented dimensions. The shared validator checks IDs, dimensions, bounds, pairwise intersections and payload. Touching faces are allowed. Utilization is the packed volume divided by the internal container volume.

What the experiment found

The hybrid tied the classical baseline on loaded volume across all five seeds for each of three cases and took longer. On IBM ibm_fez, depth-1 QAOA sampled the exact QUBO optimum for each reduced fixture, with mean hit rates ranging from 1.54% to 31.00%.

Physical scope

The model omits support, load-bearing limits, centre of gravity and securing cargo. The resulting layouts are geometric research outputs and require additional physical checks before use.

Classical and hybrid search

Deterministic First-Fit Decreasing

Packages are ordered by descending volume and then package ID. The search tries the origin and combinations of already placed upper-face coordinates in ascending z, y, x order. At each position it tries the original orientation first, followed by the remaining allowed permutations. A package that cannot fit is skipped.

Candidate generation and objective

The hybrid pool starts with the baseline placements, then adds individually feasible alternatives in package-ID rounds up to 100 candidates. Candidate coordinates come from the origin, baseline faces and positions flush with a far wall. A binary variable x_i selects candidate i.

$$E(x) = -\sum_i v_i x_i + \lambda \sum_{(i,j) \in C} x_i x_j$$

C contains overlap and same-package pairs. A pair that violates both rules is charged once. The default penalty λ is one greater than the largest volume reward. Removing either candidate from a selected conflicting pair strictly lowers the energy.

Annealing, payload repair and validation

The annealer uses seeded random bit-flip order and a geometric cooling schedule. The benchmark takes the lowest-energy state visited over 16 reads of 200 sweeps. Repair tests selected placements in decreasing volume order and keeps each one only if the shared validator accepts it with the placements already retained. This step enforces total payload.

The hybrid timer includes baseline construction for candidate seeding, candidate generation, QUBO construction, annealing, decoding, repair and validation.

Same packed volume, longer runtime

Each method used the same records for a given case. All cases used a 5.90 × 2.35 × 2.39 m container. The 15-package case adds a box that needs rotation to fit. The 20-package case reduces the payload limit to 4,000 kg. The hybrid used seeds 0–4.

Input	Packed	Volume (m³)	Utilization	Baseline (ms)	Hybrid mean (ms)
10	10	4.5140	13.62%	1.2	267
15	15	18.9395	57.15%	9.0	424
20	8	19.1535	57.80%	0.7	347

Packed count, volume and utilization were identical for the baseline and every hybrid seed. All 18 benchmark outputs had zero violations. Replaying each best run reproduced the placements and raw energy. Annealing accounted for most of the hybrid wall time.

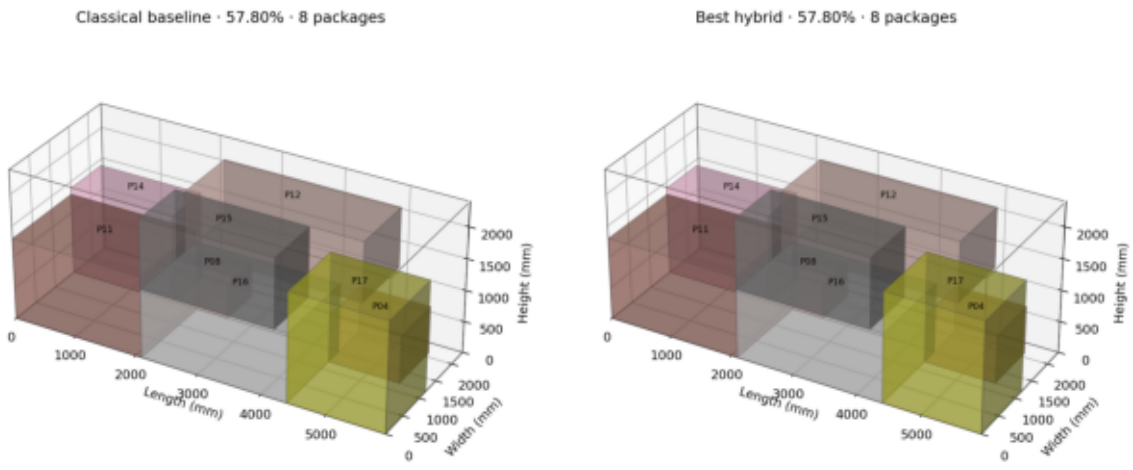


Figure 1. The baseline (left) and best hybrid run (right) for the 20-package case. Both selected the same eight package IDs and placement coordinates.

Exact references and QPU samples

H0 prepared the asymmetric logical pattern (1, 0). The intended register 01 occurred 1,013 times in 1,024 shots on ibm_fez, passing the bit-order check.

H1 tests overlap trade-offs. H2 combines rotation and duplicate-package conflicts. H3 uses 12 frozen candidates from the 10-package benchmark. Every bitstring was enumerated to verify the Ising conversion and establish the exact QUBO and payload-feasible optima.

Fixture	Ideal QAOA	IBM QPU	Annealing	QPU raw feasible
H1	32.58%	31.00%	100%	86.13%
H2	25.04%	24.96%	100%	71.46%
H3	2.25%	1.54%	100%	65.84%

Table values are raw exact-optimum hit rates except the final column. Ideal rates are statevector probabilities. Physical rates pool five 1,024-shot runs per fixture. Annealing uses the best-visited state from each of 1,024 seeded reads.

Circuit and post-processing

Depth-1 angles were chosen on an ideal expected-energy grid, frozen and transpiled for ibm_fez. Every fixture reached zero best raw energy gap and zero best repaired feasible-volume gap. Every repaired sample passed validation. Job IDs, circuit depths, two-qubit gate counts and provider timestamps are recorded in results/hardware.json.

What to test next

Comparison limits

The benchmark has three synthetic cases and a single container geometry. The candidate pool contains the baseline and excludes placements outside its finite grid. The hybrid result therefore measures search within the generated pool. The 20-package case has no full-instance optimality certificate.

The IBM fixtures are reduced tests of the QUBO-to-circuit workflow. The hardware hit rates and seeded classical annealing rates describe these fixtures and their specific sampling settings. They do not establish a hardware speed or solution-quality advantage.

Next experiment

Add tightly constrained cases with known full-instance optima. Compare the best feasible solution obtainable from each candidate pool with the full optimum to separate candidate coverage from search quality. Vary the pool cap and record runtime and repair removals. Add support and load-distribution constraints before evaluating operational datasets.

Reproduction

The README gives clean-environment setup and commands. benchmark.py regenerates the CSV, JSON and 3D plots. hardware_fixtures.py recomputes the frozen fixture reference. qpu_benchmark.py can submit new runs or retrieve recorded jobs using the saved IBM account.

Project evidence: results/benchmark.csv, results/benchmark.json, results/hardware.json, data/hardware_reference.json and the accompanying 3D comparisons.